

A estrutura na prática

Sim, existem **dois projetos separados** no Android Studio. Pensa assim:

```
ExpressMe-Watch/      ← projeto Wear OS (relógio)
ExpressMe-Companion/  ← projeto Android normal (celular)
```

São duas pastas, dois APKs diferentes, dois apps distintos. O que os conecta **não é o código** — é o Firebase. Os dois projetos apontam para o mesmo projeto no Firebase, e é por isso que conseguem se comunicar. O Firebase é o elo entre eles.

Como o Firebase fica nos dois projetos

Quando você cria um projeto no Firebase Console, ele gera um arquivo chamado `google-services.json`. Você baixa esse arquivo e coloca dentro de cada projeto no Android Studio. É literalmente um arquivo de configuração que diz "esse app pertence a esse projeto Firebase". Os dois apps têm o mesmo arquivo, por isso se entendem.

```
ExpressMe-Watch/
├── app/
│   └── google-services.json ← mesmo arquivo Firebase
ExpressMe-Companion/
├── app/
│   └── google-services.json ← mesmo arquivo Firebase
```

O plano de desenvolvimento completo

Vou dividir em fases reais, na ordem que um desenvolvedor faria.

Fase 0 — Configuração (fazer uma vez, antes de tudo)

1. Criar o projeto no Firebase Acessa `console.firebase.google.com`, cria um projeto chamado "ExpressMe". Dentro dele você registra dois apps: um com o package name do projeto do relógio, outro com o package name do projeto do celular. O Firebase gera um `google-services.json` para cada um.

2. Criar os dois projetos no Android Studio Abre o Android Studio duas vezes em pastas diferentes. No primeiro cria um novo projeto com template "Wear OS Blank Activity" — esse é o app do relógio. No segundo cria um projeto com template "Empty Activity" normal — esse é o app companion. Define o mesmo package prefix nos dois, por exemplo `com.expressme.watch` e `com.expressme.companion`.

3. Colocar o google-services.json em cada projeto Arrasta o arquivo correspondente para dentro da pasta `app/` de cada projeto. A partir daí os dois projetos já estão conectados ao Firebase.

Fase 1 — App do relógio (2 a 3 semanas)

Trabalha só no projeto `ExpressMe-Watch`. O celular nem entra ainda.

Passo 1 — Subir o emulador Wear OS No Android Studio vai em Device Manager, cria um AVD do tipo Wear OS Large Round com API 30 ou superior. Sobe o emulador e confirma que aparece a tela do relógio.

Passo 2 — Criar a tela dos 4 botões Usando Jetpack Compose for Wear OS, cria a grade 2x2 com os quatro botões. Cada botão tem cor, ícone e label. Testa no emulador — os botões precisam aparecer e responder ao toque.

Passo 3 — Adicionar vibração e áudio Quando um botão é pressionado, dispara a vibração via `VibratorManager` e reproduz o arquivo `.mp3` correspondente via `MediaPlayer`. Os arquivos de áudio ficam na pasta `res/raw/` do projeto.

Passo 4 — Adicionar a tela de confirmação Após o toque, exibe por 2 segundos uma tela com a cor do botão e um ícone de confirmação, depois volta automaticamente.

Passo 5 — Integrar o Firebase no relógio Adiciona a dependência do Firebase Messaging no `build.gradle` do projeto do relógio. Cria um serviço que, quando um botão é pressionado, envia uma mensagem ao Firebase com o tipo de emoção e o horário.

Ao final dessa fase você tem o app do relógio funcionando completo no emulador, com tudo exceto a notificação chegando no celular — isso vem na próxima fase.

Fase 2 — App companion (1 a 2 semanas)

Agora abre o projeto `ExpressMe-Companion`. O relógio já está pronto.

Passo 1 — Receber a notificação do Firebase Cria um `FirebaseMessagingService` no app companion. Ele fica escutando o Firebase e quando chega uma mensagem do relógio, dispara uma notificação no celular. Testa mandando uma mensagem de teste direto pelo Firebase Console — sem precisar do relógio ainda.

Passo 2 — Criar a tela principal com o feed de alertas Lista simples em ordem cronológica. Cada item tem ícone da emoção, mensagem e horário. As cores seguem as mesmas do relógio. Armazena os alertas localmente com Room Database ou SharedPreferences — para o MVP, SharedPreferences é suficiente e mais simples.

Passo 3 — Criar a tela de alerta ativo Quando chega uma nova notificação, o app abre essa tela automaticamente com a emoção em destaque. Um botão "marcar como visto" fecha a tela.

Passo 4 — Testar o fluxo completo Com o emulador do relógio aberto e um celular Android real com o app companion instalado, pressiona um botão no emulador e confirma que a notificação chega no celular. Esse é o momento mais importante do projeto — quando o fluxo completo funciona pela primeira vez.

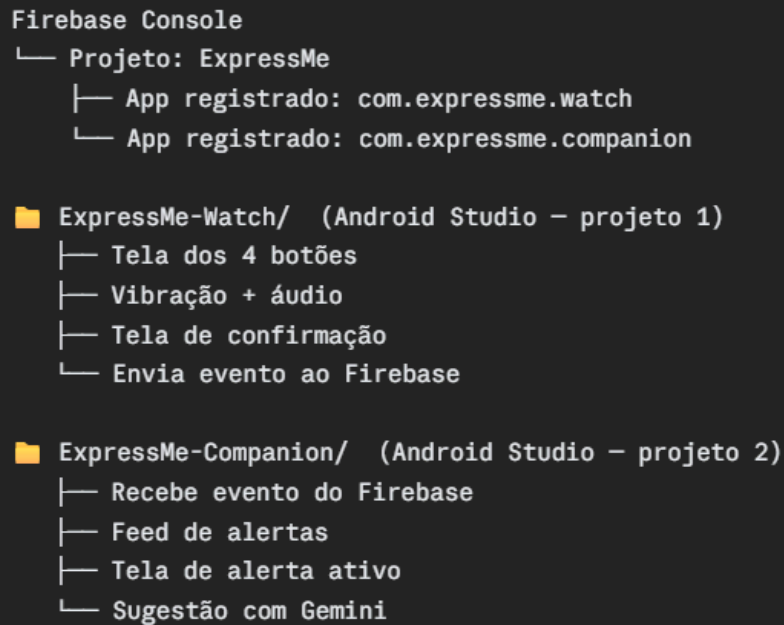
Fase 3 — Refinamento e IA (últimas semanas)

Passo 1 — Nome configurável Uma tela simples no app companion onde o cuidador digita o nome da criança. Esse nome é usado em todas as notificações.

Passo 2 — Sugestão com Gemini No app companion, cria uma lógica que conta quantas vezes o mesmo botão foi pressionado nos últimos 30 minutos. Se passar de 3, chama a Gemini API com um prompt simples e exibe a sugestão ao cuidador. São cerca de 20 linhas de código e eleva muito o projeto.

Passo 3 — Instalar no relógio físico Quando o relógio estiver disponível, habilita o modo desenvolvedor nele, conecta via ADB WiFi e instala o APK direto pelo Android Studio — exatamente igual ao celular, mas sem fio.

Visualização final da estrutura



The screenshot shows the Firebase Console interface with a dark theme. At the top, it says 'Firebase Console' with a copy icon on the right. Below this, the project 'ExpressMe' is listed with two registered apps: 'com.expressme.watch' and 'com.expressme.companion'. There are two main folders: 'ExpressMe-Watch/' (labeled 'Android Studio - projeto 1') and 'ExpressMe-Companion/' (labeled 'Android Studio - projeto 2'). The 'Watch' folder contains 'Tela dos 4 botões', 'Vibração + áudio', 'Tela de confirmação', and 'Envia evento ao Firebase'. The 'Companion' folder contains 'Recebe evento do Firebase', 'Feed de alertas', 'Tela de alerta ativo', and 'Sugestão com Gemini'.

```

Firebase Console
└─ Projeto: ExpressMe
    └─ App registrado: com.expressme.watch
        └─ App registrado: com.expressme.companion

    └─ ExpressMe-Watch/ (Android Studio - projeto 1)
        └─ Tela dos 4 botões
        └─ Vibração + áudio
        └─ Tela de confirmação
        └─ Envia evento ao Firebase

    └─ ExpressMe-Companion/ (Android Studio - projeto 2)
        └─ Recebe evento do Firebase
        └─ Feed de alertas
        └─ Tela de alerta ativo
        └─ Sugestão com Gemini

```
